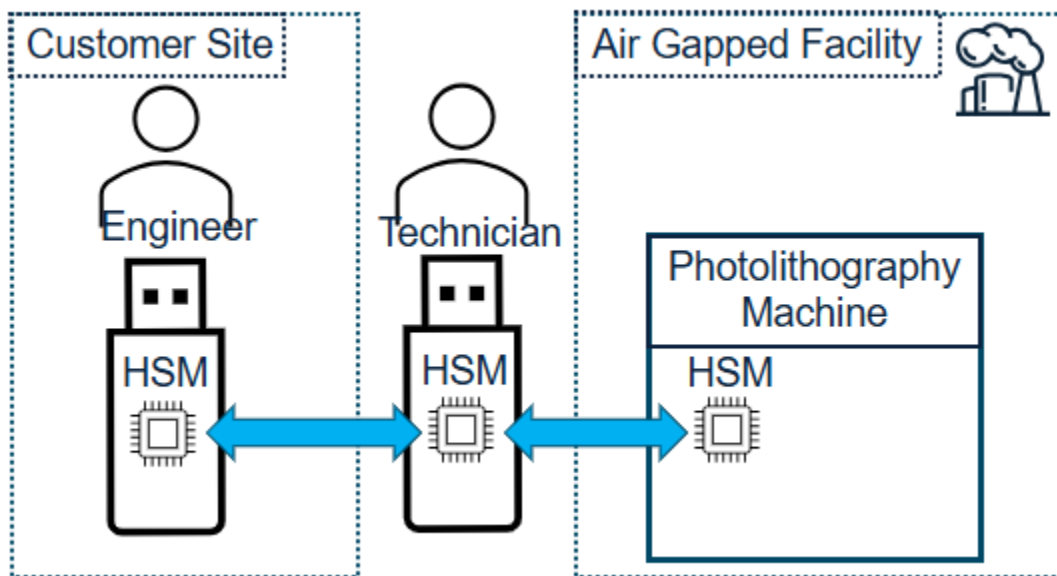


Design Documentation

UMass Amherst

System Overview

A chip fabrication company has to protect its proprietary data. They are creating a secure file storage/transfer system to ensure data confidentiality and integrity. This system contains an embedded security subcomponent for this purpose, a permissioned hardware security module (HSM). The HSM has a management interface over which a user (host computer) can list, read, or write files. It will also have a transfer interface, over which two HSMs can transfer files. This HSM must communicate with the other HSMs per the Functional Requirements, and adhere to the Security Requirements as well.



The system consists of 3 HSMs, an Engineer, Technician, and Photolithography machine. These HSMs communicate with each other for the ultimate goal of chip fabrication via the Photolithography Machine. This communication is authenticated via permission groups, in which every file belongs to a particular group. HSMs contain the permission data for a particular group. Permissions are categorized as follows: Receive, Write, and Read.

Threat Modeling

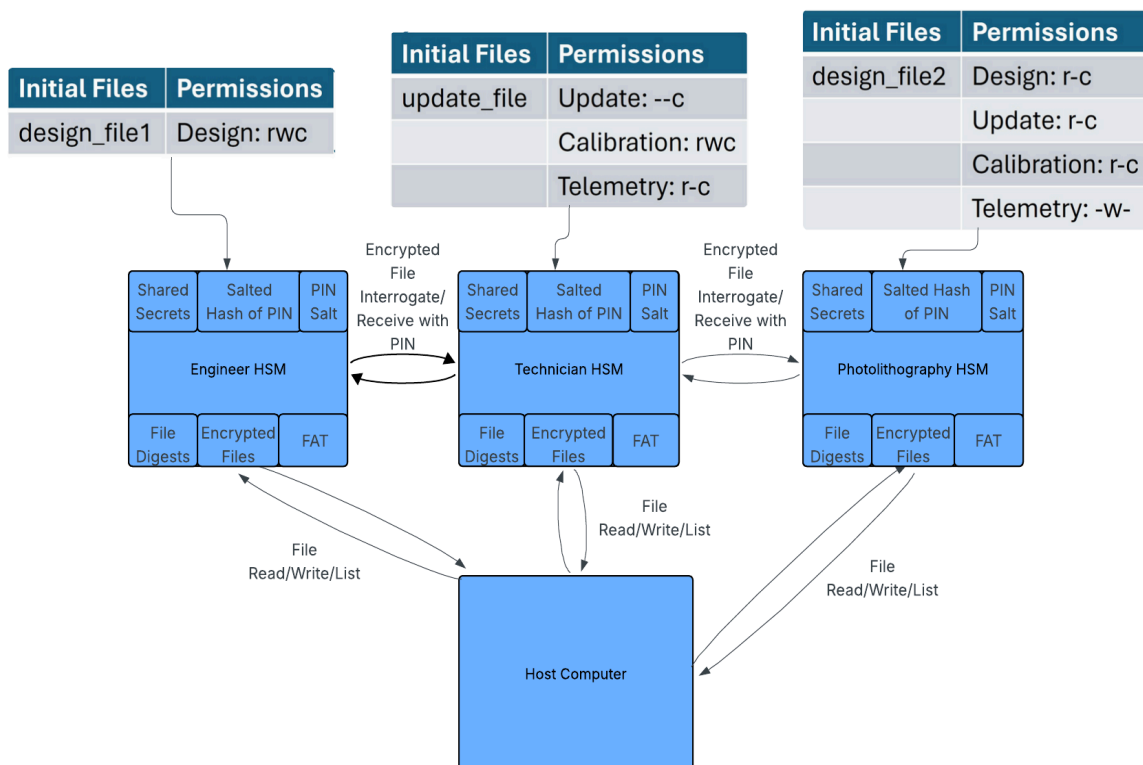
The goal is to secure a file storage/transfer system for proprietary chip fabrication data against attackers. The attacker is a malicious technician within the organization with unrestricted physical access to the Technician HSM and Engineer HSM, as well as remote access to the Photolithography HSM via the transfer interface. There are 5 attack scenarios the technician can carry out:

1. **Receive Design Files Without Permission:** The technician has physical access to the Technician HSM. This particular HSM does not have permissions to receive the design files contained within the other HSMs. The goal of the attacker is to subvert the system such that either of the HSMs send the design files to the technician.
2. **Reading Firmware Updates Without Permission:** The goal of the attacker is to extract firmware update files to learn how the Photolithography Machine functions. The Technician HSM contains a firmware update file for which the technician does not have read permissions.
3. **Reading Proprietary Designs Without Permission:** The goal of the attacker is to recover proprietary design data from the Engineer HSM and Photolithography HSM. Both of these HSMs contain a design file that the Technician HSM does not have read permissions for.
4. **Compromising Photolithography Machine Without Authorization:** The goal of the attacker is to compromise the Photolithography Machine by pushing a corrupted firmware update, resulting in maintenance costs and possibly permanent damage. The machine must accept this update as valid. The contents of the file do not matter, as long as the file does not match the original update file. The attacker does not know the PIN for the Photolithography Machine and only has access over the transfer interface.
5. **Inserting Backdoored Designs Without Authorization:** The goal of the attacker is to insert malicious data into the fabrication process. An arbitrary design file must be pushed to the Photolithography Machine. The machine must accept the file as valid. The attacker does not know the PIN for the Photolithography Machine and only has access over the transfer interface.

	Technician	Engineer	Photolithography Machine
PIN			
Physical Access			
Management Interface			
Transfer Interface			

- Attacker has unrestricted access
- Attacker has some access
- Attacker has no access

High Level Design



Functional Requirements

There are six required functional elements of the HSM firmware. Each element corresponds to a command issued by the host tools:

List Files

When a host issues a List Files command over the management interface, the firmware enumerates all stored file entries in persistent storage. For each file, the firmware retrieves its UUID, name, size, and permission group. This metadata is then serialized into the expected response format and returned to the host.

Completed in under 500 ms.

Read a File from the HSM

Upon receiving a Read File command, the file contents are read from the specified slot, which corresponds to a location in nonvolatile storage. These contents are then transmitted to the host over the management interface. Reads are performed in fixed-size chunks to accommodate interface limitations.

Completed in under 3000 ms.

Write a File to the HSM

The Write File command causes the firmware to allocate a new file entry in persistent storage. The file's metadata, including UUID, name, and permission group, is recorded prior to writing the file contents. The firmware accepts the file data from the host in sequential blocks and commits the completed file to storage.

Completed in under 3000 ms.

Listen for Messages from a Neighboring HSM

The firmware continuously monitors the transfer interface for incoming messages from neighboring HSMs. Incoming messages are parsed and dispatched to the appropriate handler based on message type without interrupting management interface operation.

Interrogate Files from a Neighboring HSM

When instructed by the host to interrogate a neighboring HSM, the firmware sends a metadata request over the transfer interface. The response is parsed and relayed to the host, allowing the host tool to display file metadata without transferring file contents.

Completed in under 1000 ms.

Receive a File from a Neighboring HSM

For file reception, the firmware accepts an incoming transfer over the transfer interface and stores the received file in persistent memory. Metadata is recorded prior to content storage, and the file becomes available for subsequent host operations once the transfer completes.

Completed in under 3000 ms.

Cryptographic Design

Our cryptographic design utilizes a hybrid form of cryptography to ensure the confidentiality, integrity, and authenticity of our filesystem and the transfer of files between HSMs.

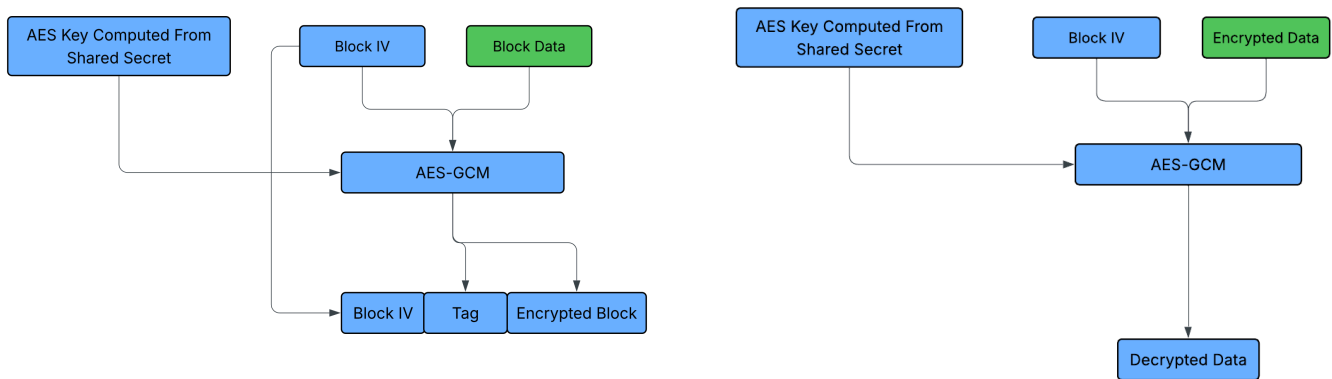
Cryptographic Primitives:

Asymmetric encryption is performed using Elliptic Curve Cryptography over Curve25519. This provides us with smaller key sizes than RSA.

Symmetric encryption is performed using standard 128-bit AES-GCM (Advanced Encryption Standard Galois Counter Mode), which securely encrypts file contents while also providing an authentication tag to ensure integrity of the encrypted content and any additional data.

When encrypting files asymmetrically, we use a hybrid form of encryption using a modified form of Elliptic-Curve Diffie-Hellman Key Exchange (ECDH). The contents of each file are encrypted with a symmetric key. This symmetric key is generated by deriving a shared secret using ECDH between a local secret key on the HSM and the recipient's public key. This shared secret is then turned into an AES key through a HMAC Key Derivation Function (HKDF) which can be used as the symmetric key. The symmetrically encrypted contents are sent along with the public key that corresponds to the local secret key. This public key can then be combined with the recipient's private key in a second round of Diffie-Hellman to derive this shared symmetric key. Finally, this symmetric key can be used to decrypt the contents.

To ensure file integrity we use Edwards Curve Digital Signature Algorithm (EdDSA) to sign and verify digests of file contents.



Our implementation uses Rust crates `aes_gcm`, `p256::ecdsa`, and `x25519_dalek`.

Permissions:

We've devised a permission system built upon the Elliptic Curve Integrated Encryption Scheme (ECIES). Our design makes extensive use of both asymmetric and symmetric encryption.

Each HSM is issued a set of public-private key pairs for each permission group. For example, if the permission groups are Telemetry, Photolithography, and Maintenance, each HSM would be issued 3 sets of public-private key pairs. These sets are composed of a public-private key pair for each permission type (read, write, and receive).

Let Key-pairs be a set of potential permission groups:

$$\text{Key-pairs} = \{G_T, G_P, G_M\}$$

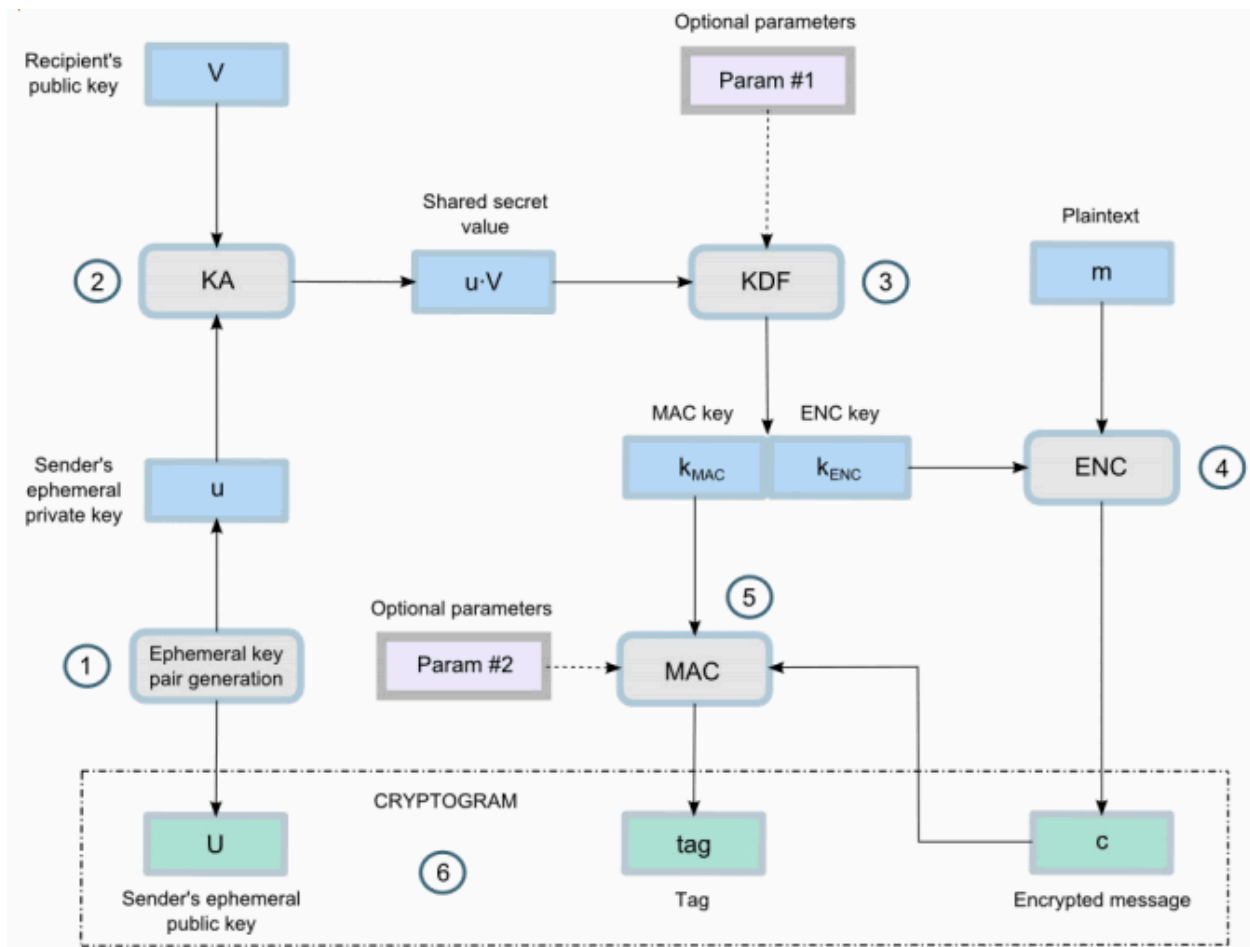
Each permission group G contains a public-private key pair corresponding to the type of permission (**R**ead, **W**rite, and **R**e**C**eive):

$$G = \{KP_R, KP_W, KP_C\}$$

$$KP = \{PUB, PRIV\}$$

All HSMs are issued a copy of the public key for every permission type (Read, Write, or Receive) within every permission group (Telemetry, Maintenance, etc.). However, an HSM is only issued a private key if it has that permission type within that permission group.

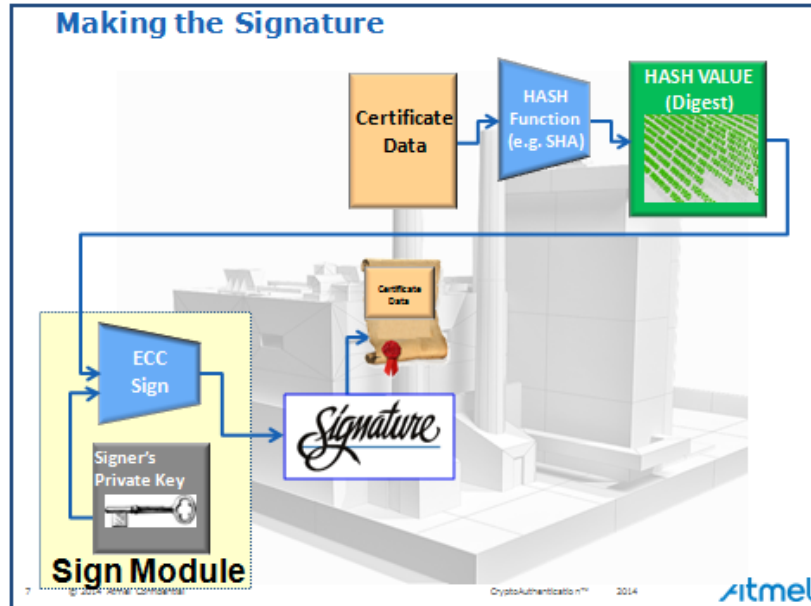
For example, if an HSM only has the write permission for the telemetry group, it will be issued a private key that corresponds to the write permission for the telemetry group and will not be issued any other private keys.



Writing Files:

The writing HSM encrypts a plaintext file's contents with the group's public read key. This allows an HSM with read permission for that group to decrypt and read the file.

To ensure that only HSMs with valid permissions can write files we use a cryptographic signature generated with the group's private write key. This can only be done by HSMs with valid write permissions. The HSM creates a hash of the file contents and then signs it using its private write key to produce this signature. Signatures are stored in file metadata.



Reading Files:

Since all files are encrypted by the HSM that writes them using the public key corresponding to the read permission of that group, an HSM in that group can read the file by decrypting it with the group's private read key.

For example, the contents of a file belonging to the Photolithography group will get encrypted with the Photolithography group's public read key. For an HSM belonging to the Photolithography group with read permissions, the HSM will use the group's private read key to decrypt and read the file.

To verify that an authorized HSM created the file, the HSM uses the public write key to decrypt the file's hash and then compares it against its own hash of the file. If the hashes match, the HSM can guarantee that it was written by an HSM with valid permissions.

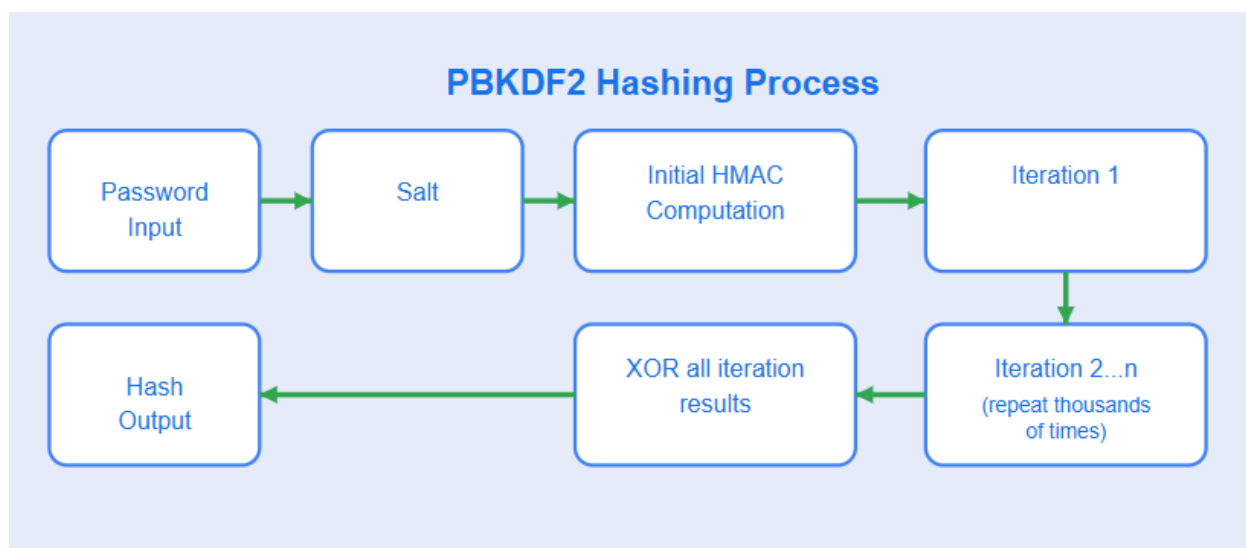
Receive File:

If an HSM has this permission, it is allowed to ask for and receive files from this group from another HSM over the transfer interface. The receive permission structure functions as follows: for any file or file metadata, the sender encrypts the content using the receiver public key, and sends this encrypted content over the transfer interface along with the group ID as plaintext. The receiver retrieves the receive private key corresponding to this group and attempts to decrypt the contents. Upon decryption, the receiver verifies the cryptographic signature of the file to ensure it has not been modified. If successful, it proceeds to write the file into a slot in the filesystem, otherwise the file is expunged.

PIN Authentication:

To ensure that only authorized users can use an HSM, we've implemented authentication using a 6 digit hexadecimal pin (per MITRE's specifications).

To avoid a brute force attack, we use PBKDF2 (Password Based Key Derivation Function) to create a hash of the pin which is then stored securely in memory. This slows down the pin check as the hash takes time to compute. This ensures that the authentication operation takes a sufficient amount of time, and also takes a constant amount of time whether the pin is correct or incorrect to prevent leaking information about the authentication through a timing side-channel.



Random Number Generation:

Performant and unpredictable random numbers are crucial to a secure implementation of AES-GCM. Many attacks against AES rely not on breaking the underlying cryptography, but by exploiting insecure random number generation. Our Cryptographically Secure Pseudorandom Number Generator is constructed by generating a seed value from the onboard True Random Number Generator (TRNG) which sources entropy from unpredictable electrical noise within the chip. This seed value is used to initialize a ChaCha20 stream cipher, the output of which is a performant and secure source of random numbers.

Security Requirements

Security Requirement 1

An attacker should not be able to perform any file action without a validly provisioned HSM with the permissions to perform that action on files belonging to that group.

The attacker should not be able to read files from an HSM without the read permission.

The attacker should not be able to create files without the write permission. The attacker should not be able to receive files from other HSMs without the receive permission. The receive permission also applies to interrogate, meaning that the interrogated device should only return metadata about files for which the requesting device has the receive permission.

Our cryptographic design addresses permission enforcement as follows:

Writing Files: We ensure that only HSMs with valid permissions can write files by generating a signature with the private write key for the particular group. This is only possible with valid write permissions. The HSM hashes the file content and encrypts it using the private write key that was used to generate the signature.

Reading Files: All files are encrypted by the HSM that writes them using the public key corresponding to the read permission of that group, so only an HSM in that group can read the file by decrypting it with the group's private read key.

Receiving Files: We require receive permissions by having the sender HSM sign with the receive public key for the message and send along the group ID as plaintext, and the receiver HSM retrieves the receive private key that corresponds to the group ID and attempts to decrypt this message. Upon decryption, the receiver verifies the cryptographic signature to ensure file integrity. If successful, the file is written into the slot, otherwise the file is erased.

Security Requirement 2

No PIN-protected action should be able to be completed by a user without prior knowledge of the PIN. This includes confidentiality of the PIN. The HSM should not expose information about its PIN to any unauthorized user.

We devised a scheme in which we generate a pin salt and pin hash during the secrets generation phase. For each PIN-protected command, we hash the user-provided pin with the previously generated pin salt, and compare this newly derived hash with the stored

hash. If it succeeds, the command will execute as specified, otherwise, the rest of the command will not execute and return back to the main loop.

Security Requirement 3

An HSM should not successfully receive any file that was not generated by another valid HSM with write permissions for that group. This includes protecting file integrity from being compromised in any way by an attacker.

Upon receiving a file from a neighboring HSM, an HSM will verify the hash sent with the file against the group's verification key. If the signature matches the digest, the file is deemed valid and written to a slot. Otherwise, the file is removed.

Defenses

Management interface defenses

All management operations that modify or disclose sensitive information require successful PIN authentication. The firmware enforces strict command sequencing and rejects malformed or out-of-order requests. Input buffers are length-checked and validated prior to parsing to prevent memory corruption and command injection attacks.

Transfer interface defenses

File transfers are protected by a mutual authentication protocol that verifies the identity and permission group of the sending HSM. Files are accepted only if the sender's permissions satisfy the receiving HSM's policy.

Storage defenses

Files are encrypted before being written to nonvolatile storage. Encryption keys are derived from device-specific secrets and are never exposed through external interfaces.

Authentication & authorization

Each file operation is independently authorized against the file's permission group and the caller's authenticated identity.

Firmware integrity & state safety

The firmware will strictly validate all commands issued, refusing to process any malformed commands.

Physical & Side-Channel Attack Mitigations

As attackers may have full physical access to the HSM board and visibility of execution, we are limited in our ability to prevent physical and side-channel attacks. Where possible, the firmware avoids data-dependent error behavior, such as short-circuit evaluation/termination. This increases the difficulty of certain side-channel attacks, but is not resilient to power analysis or fault injection.

Optimizations

AES Hardware Acceleration

The MSPM0L2228 comes equipped with a hardware AES accelerator that supports AES-GCM. Performing encryption and decryption operations using the AES accelerator dramatically improves run-time performance compared to performing AES operations in software.

Pre-Computing Diffie-Hellman Key Exchange

The MSPM0L2228 does not have accelerated elliptic curve hardware or even accelerated multiplication instructions. As such, asymmetric operations such as ECDH key exchange are incredibly time and CPU intensive. To reduce this load, a local secret key and corresponding private key is generated for an HSM at compile time. ECDH key exchange is performed at compile time between the read and receive public key for each permission group in the deployment. The derived shared secret is then stored as a secret which can be quickly accessed without any computation. While re-using static secrets does eliminate forward secrecy, we determined this was not a strictly necessary requirement. The time-savings from this measure were around 1000 ms.

In-Place Encryption and Decryption

The MSPM0L2228 has very limited RAM at only 32 KB. With each file being 8 KB, we are significantly limited in the number of copies of a file that can exist on the stack concurrently. To reduce memory overhead, all encryption and decryption operations are performed in place.

References

https://rules.ectf.mitre.org/2026/flags/attack_flags.html

<https://cryptobook.nakov.com/asymmetric-key-ciphers/ecies-public-key-encryption>

<https://atmelcorporation.wordpress.com/tag/ecdsa-signature-protocol/>

<https://medium.com/@aannkkiittaa/how-password-hashing-works-pbkdf2-argon2-more-95cee0cd7c4a>